Vendor Portal — Product & Technical Spec (v1.0, final for build)

Oct 14, 2026 · Owner: Mohit Bansal (Solfin)

> **For the coding agent:** this file is the single source of truth. Start with the **Build process for the coding agents** section — an architect review comes first, then implementation and Playwright testing run in parallel. Build every row marked **MVP**. Phase 2 and Phase 3 items are listed only so the schema and code leave room for them — do not build them now. Where a value is marked *default* in the last section, implement it as stated but keep it configurable (DB table or env var), never hard-coded. If anything here conflicts, the "Decisions and build defaults" table at the end wins.

Proposal: build the vendor portal fully in-house as **one Django codebase**. Pages are server-rendered with Django templates; there is no separate front end and no separate API service. Data lives in PostgreSQL, and documents and invoice files in AWS S3. Vendors self-onboard, upload invoices and get email confirmations. Internal approvers (L1, L2, L3) and Finance move invoices to payment.

Build process for the coding agents

Work runs in three stages. Stage 1 must finish and be approved before any application code is written. In Stage 2, building and Playwright testing run in parallel. Stage 3 is the release gate.

Stage 1 — Architect review (one architect agent, before coding)

The architect agent reads this whole spec and produces `docs/ARCHITECTURE.md` in the repo. It writes no feature code. The document must contain:

1. **Spec review:** gaps, contradictions or risky assumptions found in this spec, each with a proposed resolution. Example: what happens to in-flight invoices when the approval matrix changes.
2. **Django app layout:** final list of apps and what each owns, confirming or adjusting the tree in "Architecture and tech stack".
3. **Model design:** every model with fields, types, constraints, indexes and relations, matching the data model table. Include the RLS policy SQL and how the middleware sets `app.vendor_id`.
4. **URL and view map:** every page for `/vendor/` and `/console/`, its view, template, required permission and the HTMX fragments it uses.

5. **State machines:** vendor and invoice statuses, the allowed transitions, who can trigger each one, and the side effects (emails, audit, notifications).
6. **Permission matrix:** role × action, generated from `core/permissions.py`.
7. **Interfaces and seams:** `EmailSender` (`solfin_api`, `console`), `parse_document()` engines (`textract`, `fake`) and storage (S3 / MinIO), so tests never need real external services.
8. **Test plan:** the Playwright suites listed below, mapped to features and acceptance criteria. Also the seed data and fixtures each suite needs (fixed users per role, two vendors A and B, sample invoices at ₹50,000 and ₹15,00,000, fake parser outputs).
9. **Work breakdown:** ordered tasks per module, with dependencies, sized so that a builder agent and a tester agent can work on the same module at the same time.

Gate: Mohit (or the lead engineer) reviews `docs/ARCHITECTURE.md` and approves it. Any change to this spec that comes out of the review is written back into this file before Stage 2 starts.

Stage 2 — Build and test in parallel

For each module in the work breakdown, two agents work at the same time from `docs/ARCHITECTURE.md`:

- **Builder agent:** implements models, migrations, services, views, forms and templates, plus pytest unit tests for services and permissions.
- **Tester agent:** writes the Playwright tests for the same module from the URL map, state machines and acceptance criteria. It writes them against the agreed URLs, form field names and `data-testid` attributes, without waiting for the build to finish. Every interactive element in a template must carry a stable `data-testid` (for example `data-testid="invoice-approve-btn"`) so the tests don't depend on styling or copy.

Rules for both:

- Both agents work against the same feature branch. A module is done only when its unit tests and its Playwright tests pass in `docker-compose.test.yml`.
- Tests use `PARSER_ENGINE=fake`, the console email backend (assert on captured emails) and MinIO for S3. No test calls Textract, the Solfin email API or real AWS.
- Each test resets the database to the seed state, so suites can run in parallel (`pytest -n auto` with pytest-xdist, one Playwright browser context per worker).
- If the tester finds a mismatch with the architecture doc, it files it as a failing test plus a note. The builder fixes the code, or the architect doc is updated; tests are never deleted to make a build pass.

Playwright suites (tests/e2e/), all required for MVP:

Suite	Covers
`test_auth.py`	Login, OTP step, wrong OTP, lockout after 5 failures, password reset, idle logout, role-based landing page
`test_vendor_onboarding.py`	Invite email captured → magic link → registration → KYC upload with fake parse pre-fill → submit → Vendor Manager sends back → resubmit → activate → welcome email
`test_kyc_review.py`	Per-document approve/reject/remarks; activation blocked until mandatory documents are approved; the inviter cannot also activate
`test_invoice_upload_parse.py`	Upload PDF → "Reading your invoice" state → fields pre-filled with low-confidence fields highlighted → edit → submit; duplicate invoice blocked; GSTIN mismatch blocked; totals mismatch blocked; file over 10 MB and wrong type rejected
`test_approval_routing.py`	₹50,000 invoice: Finance → L1 → Approved. ₹15,00,000 invoice: Finance → L1 → L2 → L3 → Approved. The right approver sees it in their queue at each step, and the right email is captured at each step
`test_maker_checker.py`	Uploader cannot approve; L1 approver cannot act again at L2; Finance cannot approve at L1–L3; bank-detail change holds payment until Finance approves
`test_query_reject.py`	Query at L2 → vendor replies → invoice returns to L2; reject at any level is final and the vendor is emailed
`test_payments.py`	Record full and partial payments with UTR and TDS; vendor sees status and payment history; payment advice email

Suite	Covers
`test_vendor_isolation.py`	Vendor A cannot open vendor B's invoices, documents, files or payment pages by URL, by guessing IDs, or through export links. Every attempt returns 404
`test_reports_exports.py`	Ageing and pending-by-approver reports show correct counts; CSV/Excel downloads have the expected columns
`test_admin_config.py`	Admin edits approval limits and the routing changes for new invoices; admin edits the document checklist; template on/off
`test_audit.py`	Every approval, status change, bank change and login appears in the audit log with actor and timestamp
`test_smoke.py`	Tagged `@smoke`: login, upload + parse, L1 approve. Runs after every deploy to UAT and prod

Stage 3 — Release gate

1. The full unit + Playwright run in CI is green, and the run report and traces are attached to the release.
2. The architect agent does a final review of the code against `docs/ARCHITECTURE.md`, covering permissions, isolation, migrations and anything that drifted, and lists the fixes needed.
3. The acceptance criteria in "Delivery plan and acceptance criteria" are ticked off.
4. Deploy to UAT, run `@smoke`, then hand over for Finance and approver sign-off.

Overview and goals

Today vendor invoices arrive by email. Finance re-keys them, chases missing documents and answers "when will I be paid" calls. The portal replaces that with one system of record, where every vendor, document and invoice has an owner and a status.

Goals for v1:

1. Vendor onboarding with KYC (PAN, GSTIN, bank, MSME) done end-to-end in the portal, with no email back-and-forth.

2. 100% of vendor invoices submitted through the portal, each linked to a vendor and a cost centre.
3. An automatic email at every state change: received, approved, rejected, query raised, paid.
4. Vendors can see invoice and payment status themselves, which cuts status-enquiry calls.
5. A full audit trail for every approval and every change to bank details.

Scope and non-goals

In scope (v1): vendor invitation and self-registration, KYC document collection and verification, vendor login with multiple users per vendor, invoice upload (PDF/image) with automatic document parsing, a maker-checker approval workflow (Finance pre-check → L1 → L2 → L3), payment recording, email notifications, an internal admin console, an audit log and basic reports.

Not in scope (v1):

- Accounting integration. There is no ledger and no push to Tally/ERP in v1. Approved invoices and payments export to CSV/Excel instead.
- Purchase orders. All invoices are non-PO in v1 and must carry a cost centre.
- Payment execution. The portal records the UTR and payment date; money moves through the bank as it does today.
- Purchase requisitions, RFQs or tendering.
- Contract lifecycle management beyond storing the signed agreement.
- Mobile apps. The portal is responsive web only.

Users, roles and permissions

There are eight roles: two on the vendor side and six internal. Vendor users only ever see their own vendor's records. This is enforced in every Django view and queryset and backed by Postgres row-level security. Internal users can hold more than one role, but the system blocks approving anything they created or already approved at an earlier level.

Role	Side	Who	Can do	Cannot do
Vendor Admin	Vendor	Primary contact at the vendor	Edit profile and KYC, add/remove vendor users, upload invoices,	See other vendors; change bank details without re-verification

Role	Side	Who	Can do	Cannot do
			reply to queries, view payments	
Vendor User	Vendor	Vendor's accounts staff	Upload invoices, reply to queries, view status	Manage users, edit KYC or bank
Vendor Manager (Procurement)	Internal	Owns vendor relationships	Invite vendors, review KYC, activate/block vendors	Approve invoices; record payments
L1 Approver	Internal	Department / cost-centre owner	Approve, reject or query invoices routed to them (all amounts start here)	Edit invoice amounts; approve own uploads
L2 Approver	Internal	Function head	Approve, reject or query invoices above the L1 limit	Act on invoices they approved at L1
L3 Approver	Internal	CFO / CEO	Final approval above the L2 limit	Act on invoices they approved at L1/L2
Finance (AP)	Internal	Accounts payable team	Finance check (GST, TDS, cost centre), mark paid with UTR, export to CSV/Excel, approve bank-detail changes	Change vendor KYC; approve at L1–L3
Admin	Internal	Tech / system owner	Users and roles, approval matrix, email templates, document	Approve invoices or KYC (segregation of duties)

Role	Side	Who	Can do	Cannot do
			checklist, integrations	

Optional ninth role, **Auditor**: read-only access to everything, including the audit log.

Feature list

The MVP covers onboarding, invoice upload with parsing, approval and emails end-to-end. Build the MVP rows now. Phase 2 and 3 rows are listed so the schema leaves room for them.

#	Module	Feature	Phase
1	Onboarding	Internal user invites a vendor by email; magic-link invite valid 7 days	MVP
2	Onboarding	Vendor self-registration form (company, contacts, address, category)	MVP
3	Onboarding	KYC upload: PAN, GST certificate, cancelled cheque, MSME/Udyam certificate, signed agreement. The parser reads the PAN, GSTIN, legal name, IFSC, account number and Udyam number and pre-fills the form	MVP
4	Onboarding	Configurable document checklist per vendor category (goods / services / professional)	MVP

#	Module	Feature	Phase
5	Onboarding	KYC review: approve, reject or send back with remarks, per document	MVP
6	Onboarding	Vendor states: Invited → Draft → Submitted → Under Review → Active / Rejected → Blocked	MVP
7	Onboarding	GSTIN and PAN auto-verification via KYC API; PAN–GSTIN name match	Phase 2
8	Onboarding	Bank account penny-drop verification	Phase 2
9	Onboarding	Document expiry tracking (agreement, certificates) with reminders 30/7 days before	Phase 2
10	Access	Email + password login, OTP on email, password reset	MVP
11	Access	Multiple users per vendor, managed by Vendor Admin	MVP
12	Access	SSO (Microsoft Entra ID) for internal users	Phase 2
13	Invoices	Upload PDF/JPG/PNG, max 10 MB, with invoice no., date, amount,	MVP

#	Module	Feature	Phase
		GST amount, cost centre (non-PO)	
14	Invoices	Duplicate check on vendor + invoice number + financial year	MVP
15	Invoices	Bulk upload (multiple invoices in one go)	Phase 2
16	Invoices	Invoice parsing: extracts supplier GSTIN, invoice no., date, PO no. (if printed), line items, taxable value, CGST/SGST/IGST and total from PDF/image, with per-field confidence. The user confirms or corrects before submit	MVP
17	Invoices	E-invoice IRN / QR validation for vendors above the e-invoicing threshold	Phase 3
18	Approval	Workflow: Finance pre-check → L1 → L2 (above L1 limit) → L3 (above L2 limit) → Finance for payment	MVP
19	Approval	Approve / reject / raise query, with mandatory comment on reject or query	MVP

#	Module	Feature	Phase
20	Approval	Query thread between approver and vendor on each invoice	MVP
21	Approval	SLA timers and escalation emails for pending approvals	Phase 2
22	Payments	Record payment: amount, date, UTR, TDS deducted; partial payments allowed	MVP
23	Payments	Vendor sees payment history and outstanding per invoice	MVP
24	Payments	MSME 45-day due-date tracking and overdue flag	Phase 2
25	Payments	Bank payment file (bulk upload format) generation	Phase 3
26	Notifications	Email on every state change (see the catalogue)	MVP
27	Notifications	In-portal notification bell	MVP
28	Notifications	WhatsApp / SMS notifications	Phase 3
29	Reports	Vendor master, invoice ageing, pending approvals by approver, payments register	MVP

#	Module	Feature	Phase
30	Reports	CSV/Excel export of every list view	MVP
31	Integrations	Export approved invoices and payments to CSV/Excel (accounting integration deferred)	MVP
32	Integrations	GSTR-2B reconciliation against uploaded invoices	Phase 3
33	Admin	User and role management (L1/L2/L3, Finance, Vendor Manager, Admin), approval matrix with amount bands, email templates, document checklist	MVP
34	Admin	Audit log of every field change, approval and login	MVP

Key workflows

Vendor onboarding

1. A Vendor Manager creates a vendor stub (name, email, category) and sends an invite. The vendor gets a magic link.
2. The vendor sets a password and fills the registration form: legal name, PAN, GSTIN(s), addresses, contacts, bank details and MSME status.
3. The vendor uploads the checklist documents for its category. The parser pre-fills PAN, GSTIN, legal name and bank fields from them, and flags any mismatch with what the vendor typed. The vendor confirms, then submits.
4. The Vendor Manager reviews each document. They either approve all and activate the vendor, or send it back with per-document remarks, which returns the vendor to Draft.

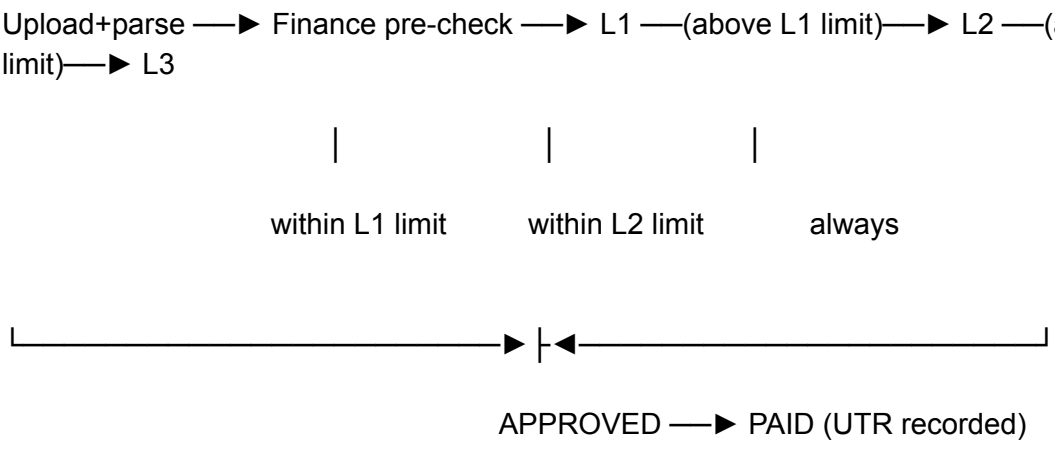
5. On activation, the vendor gets a welcome email and the invoice menu unlocks.
6. Any later change to bank details or GSTIN sends the vendor back to Under Review for that change only. Invoices keep flowing, but payments are held until it is approved.

Invoice lifecycle

1. **Upload and parse:** a vendor user uploads the invoice file. The parser extracts its fields within about 10 seconds and pre-fills the form, marking low-confidence fields in amber. The user corrects any fields and submits.
2. **Validations on submit:** the duplicate check (vendor + invoice no. + FY) runs; the GSTIN on the invoice must match the vendor master; taxable + GST must equal the total (±₹1); a cost centre is mandatory, because all invoices are non-PO in v1.
3. **Submitted:** the vendor gets an "invoice received" email with a portal reference.
4. **Finance pre-check:** Finance verifies GST and the TDS section, then forwards to L1 or raises a query.
5. **L1 approval:** the cost-centre owner approves, rejects or queries. The approval matrix decides whether the invoice stops here.
6. **L2 approval:** required when the amount is above the L1 limit.
7. **L3 approval:** required when the amount is above the L2 limit.
8. **Query** (any stage): the invoice moves to *Query Raised* and the vendor is emailed. The vendor replies or re-uploads, and the invoice returns to the stage that raised the query.
9. **Approved:** the vendor is emailed with the expected payment date, and the invoice appears in the Finance payment queue.
10. **Paid / Partly Paid:** Finance records the UTR, date, amount and TDS. The vendor gets a payment advice email.
11. **Rejected** at any stage is final. The vendor must upload a new invoice.

Example limits, configurable in the approval matrix: L1 up to ₹1 lakh, L2 up to ₹10 lakh, L3 above ₹10 lakh.

Approval routing (each invoice climbs only as many levels as its amount needs):



Any stage: QUERY → vendor replies → back to the same stage. REJECT → final.

Invoice statuses: `DRAFT`, `SUBMITTED`, `FIN_PRECHECK`, `PENDING_L1`, `PENDING_L2`, `PENDING_L3`, `QUERY_RAISED`, `APPROVED`, `PARTLY_PAID`, `PAID`, `REJECTED`.

Data model

PostgreSQL 16, one schema `vp`. Every table has a UUID primary key, `created_at`, `updated_at`, `created_by` and `updated_by`. Vendor-owned tables carry `vendor_id` for row-level security. Money is stored as `numeric(14,2)`, in INR.

Table	Key columns	Notes
users	email (unique), name, phone, password_hash (argon2id), user_type (vendor / internal), status, mfa_enabled, last_login_at	One table for both sides (custom Django user model)
roles, user_roles	role code (VENDOR_ADMIN, VENDOR_USER, VENDOR_MANAGER, L1, L2, L3, FINANCE, ADMIN, AUDITOR); user_id, role_id, scope (cost_centre_id nullable)	L1 can be scoped to cost centres
vendors	vendor_code, legal_name, trade_name, category, pan, constitution, msme_status, udyam_no, msme_type, status, owner_manager_id, activated_at	Status: INVITED, DRAFT, SUBMITTED, UNDER_REVIEW, ACTIVE, REJECTED, BLOCKED
vendor_users	vendor_id, user_id, is_admin	Links logins to a vendor
vendor_gstins	vendor_id, gstin, state_code, address, is_primary, verified_at	Many per vendor
vendor_bank_accounts	vendor_id, holder_name, account_no_enc,	Versioned; changes need Finance approval

Table	Key columns	Notes
	account_last4, ifsc, bank_name, status, verified_at, approved_by	
document_types	code, name, applies_to (categories), mandatory, has_expiry, parser_profile	The configurable checklist
files	s3_key, bucket, mime, size_bytes, sha256, uploaded_by, scan_status	Every upload; sha256 catches duplicate files
vendor_documents	vendor_id, document_type_id, file_id, valid_from, valid_to, status, reviewer_id, remarks	One row per KYC document
parse_jobs	file_id, kind (INVOICE / PAN / GST / CHEQUE / UDYAM), engine, status, raw_json, extracted_json, confidence, error, duration_ms	Output of the document parser
cost_centres	code, name, l1_owner_id	Routing for L1
purchase_orders	po_no, vendor_id, cost_centre_id, amount, consumed_amount, valid_till, status	Phase 2 only; not used in the MVP (all invoices are non-PO)
invoices	vendor_id, gstin_id, invoice_no, invoice_date, fy, po_id (nullable, unused in v1), cost_centre_id, taxable, cgst, sgst, igst, total, tds_section, tds_amount, file_id, parse_job_id, status, current_level, due_date	Unique (vendor_id, invoice_no, fy)
invoice_line_items	invoice_id, description, hsn_sac, qty, rate, amount, tax_rate	Filled by the parser, editable

Table	Key columns	Notes
approval_matrix	category, cost_centre_id (nullable), l1_max, l2_max, requires_l3_above	Amount bands per level
approval_actions	invoice_id, level (FIN_PRE, L1, L2, L3), actor_id, action (APPROVE / REJECT / QUERY), comment, acted_at	Full approval history
invoice_messages	invoice_id, from_user_id, body, file_id, created_at	Query thread
payments	invoice_id, amount, paid_on, utr, tds_deducted, recorded_by	Many per invoice
notifications	user_id, template_code, payload, channel, status, sent_at, error	Email + in-app log
audit_log	actor_id, entity, entity_id, action, before_json, after_json, ip, user_agent, at	Append-only; no UPDATE/DELETE grants

Schema changes go through Django migrations in the repo. Seed scripts (management commands) create the roles, document types, a sample cost centre and a sample approval matrix.

Architecture and tech stack

One Django project, one repo, one application image. Django renders every page with its own templates; there is no React/Next.js and no separate REST API for the UI. The same image runs three ways under docker-compose:

- **app:** Django on gunicorn. It serves both portals: `/vendor/...` for vendor users and `/console/...` for internal users (Vendor Manager, L1/L2/L3, Finance, Admin). Django admin lives at `/sysadmin/` for System Admin configuration only.
- **worker:** Celery, for parsing, email and exports.
- **beat:** Celery beat, for reminders and digests.

Postgres is the only system of record. S3 holds every file. Redis is the Celery broker and cache. Nginx sits in front, serves `/static/`, and proxies everything else to Django.

```
vendor-portal/
  manage.py
  config/                settings (base/local/uat/prod), urls, celery, wsgi
  apps/
    accounts/            custom User, roles, OTP login, password reset, session auth
    vendors/             vendors, GSTINs, bank accounts, KYC documents, onboarding views
    invoices/            invoices, line items, approval workflow, queries, payments
    parsing/             parse jobs, engine adapters (textract, fake), parse_document()
    notify/              email templates, EmailSender (solfin_api, console), in-app bell
    reports/             list views, CSV/Excel exports
    audit/               append-only audit log, middleware
    core/                vendor-isolation middleware, RLS helpers, permissions, base models
  templates/
    base.html            layout, nav, messages, notification bell
    vendor/              vendor portal pages
    console/             internal portal pages
    emails/              email templates (HTML + text)
    partials/            HTMX fragments (tables, status badges, parse results)
  static/                Tailwind build output, htmx.min.js, small vanilla JS
  tests/
```

  unit/ pytest + pytest-django

  e2e/ Playwright (pytest-playwright)

docker/

  Dockerfile

  docker-compose.yml

  docker-compose.test.yml

  nginx/nginx.conf

.env.example

Layer	**Choice**	**Why**
Framework	Django 5 (Python 3.12), one project	One codebase, one deploy; auth, ORM, forms, templates and admin built in
UI	Django templates + Django forms (django-widget-tweaks for styling), Tailwind CSS via the standalone CLI (no Node runtime in prod), HTMX for partial updates (parse status polling, approve/reject without full reload, query thread), minimal vanilla JS	Server-rendered; no separate front end to build or deploy
ORM / migrations	Django ORM + Django migrations	
Vendor isolation	A `VendorScopedManager` filters every vendor-owned queryset by the logged-in vendor, plus Postgres RLS. Middleware runs `SET LOCAL app.vendor_id` inside `ATOMIC_REQUESTS`	Data is protected even if one view forgets a filter

Layer	Choice	Why
Database	PostgreSQL 16	
Files	AWS S3 private bucket via django-storages + boto3. Upload forms post to Django, which streams the file to S3. Downloads go through a permission-checked view that redirects to a 5-minute pre-signed URL	No public file URLs
Jobs	Celery + Redis; Celery beat for SLA reminders and daily digests	Retries and scheduling built in
Auth	Django session auth (secure, HttpOnly, SameSite=Lax cookies) with Argon2 password hasher, email OTP as a second step, django-axes for lockout; Entra ID OIDC for internal users in Phase 2	No JWTs needed with server-rendered pages
RBAC	Django groups (VENDOR_ADMIN, VENDOR_USER, VENDOR_MANAGER, L1, L2, L3, FINANCE, ADMIN, AUDITOR) + one permission map in `core/permissions.py`, enforced by view mixins/decorators and used by templates to show or hide actions	One source of truth for rights
Workflow	Explicit state-machine service in `invoices/services.py` (django-fsm or plain functions); views never	Rules live in `approval_matrix` data, not code

Layer	Choice	Why
	change status directly. Every transition writes `approval_actions` + `audit_log` in one transaction	
Email	Solfin's custom email API behind an `EmailSender` interface, called from a Celery task with retries; templates rendered by Django	Per Solfin's setup
Document parsing	See the subsection below	MVP feature
Reverse proxy	Nginx container: TLS, gzip, static files, 10 MB upload limit, rate limits on login and upload	
Observability	Structured JSON logs, Sentry for Django, `/healthz` endpoint, Flower for Celery (internal only)	
CI	GitHub Actions: ruff + pytest (unit), Playwright e2e against `docker-compose.test.yml`, build and push the image	

Document parsing

Parsing runs asynchronously, so an upload never blocks on it. The steps:

1. The upload form posts to Django, which stores the file in S3, creates `files` and `parse_jobs` rows and enqueues a Celery job. The page shows a "Reading your invoice…" state and polls with HTMX every 2 seconds until the job finishes.
2. The worker runs a virus scan (ClamAV), then OCR + extraction:
    - **AWS Textract AnalyzeExpense** for invoices: header fields and line items.
    - **Textract AnalyzeDocument** (forms/queries) for PAN, GST certificate, cheque and Udyam.

3. An LLM step normalises the OCR output into a fixed JSON schema per document type (Pydantic-validated). It fills in GST split and HSN/SAC, and reads the e-invoice QR when present.
4. Rule checks run on the result: GSTIN and PAN format and checksum, PAN embedded in the GSTIN, totals reconcile, invoice date within FY. Each field gets a confidence score.
5. The result is saved to `parse_jobs.extracted_json`. The UI shows pre-filled fields, with low-confidence ones in amber. A human always confirms before submit; parsed values are never auto-submitted.

The parser sits behind one interface (`parse_document(kind, file_id)`), so the engine can be swapped without touching the rest of the system. Engines are selected by the `PARSER_ENGINE` env var: `textract` (default for UAT/prod), `fake` (returns deterministic fixtures, used by tests and local dev), and later optionally self-hosted PaddleOCR/docTR plus an LLM.

Email and notification catalogue

All emails go through Solfin's custom email API. Django renders each template, keyed by `template_code`, and a Celery task posts it to the API. Failed sends retry 3 times with backoff, and every send and response is logged in `notifications`. Every email carries the vendor code, the invoice reference and a deep link to vendor.solfin.co.in. Admins can switch individual templates on or off.

Trigger	To	CC	Email
Vendor invited	Vendor contact	Vendor Manager	Invitation with magic link, expiry date, checklist
Registration submitted	Vendor Manager	—	New vendor awaiting KYC review
KYC sent back	Vendor Admin	—	Per-document remarks, link to fix
Vendor activated	All vendor users	Vendor Manager	Welcome, vendor code, how to upload invoices
Vendor blocked	Vendor Admin	Vendor Manager	Reason and contact

Trigger	To	CC	Email
Bank details change requested	Vendor Manager, Finance	Vendor Admin	Change awaiting verification; payments on hold
Invoice submitted	Uploader + Vendor Admin	AP inbox	Receipt confirmation with portal reference
Approval pending	Current approver(s)	—	Invoice summary, approve/reject link
Query raised	Uploader + Vendor Admin	Approver	Query text, reply link
Vendor replied	Approver who raised it	—	Reply text, link
Invoice approved	Uploader + Vendor Admin	—	Approved, expected payment date
Invoice rejected	Uploader + Vendor Admin	—	Reason, re-upload link
Payment recorded	Vendor Admin	Uploader	Payment advice: amount, UTR, date, TDS
SLA breach (Phase 2)	Approver	Approver's manager	Pending more than N days
Document expiring (Phase 2)	Vendor Admin	Vendor Manager	Upload a renewed document
Daily digest	Approvers, AP	—	Pending count and oldest items

Security, data isolation, compliance and audit

The biggest risk is one vendor seeing another vendor's data, so isolation is tested before anything else ships.

- **Isolation:** every vendor-owned table has `vendor_id`. Middleware sets `app.vendor_id` per request, and Postgres RLS policies filter on it, so a missed filter

still cannot leak data. A Playwright suite logs in as vendor A and tries to reach vendor B's invoices, documents and files through every URL (by guessing IDs too).

- **Files:** private S3 bucket with block-public-access on. A download view checks the caller can see the parent record, then redirects to a 5-minute pre-signed URL. Uploads are limited to PDF/JPG/PNG, up to 10 MB, and are ClamAV-scanned before parsing or display.
- **Auth:** Django sessions (Secure, HttpOnly, SameSite=Lax), Argon2 hashing, email OTP for vendors, lockout after 5 failed attempts (django-axes), idle logout after 30 minutes, Django CSRF protection on every form, security headers (HSTS, CSP, X-Frame-Options).
- **RBAC:** every view declares its permission. Approve actions also check that the invoice's `current_level` matches the caller's role, that the caller is within their amount band, and that they are not the uploader or an earlier approver of that invoice.
- **Sensitive fields:** bank account number and PAN are encrypted at the application level with a KMS data key. Only the last 4 digits show, except to Finance.
- **Maker-checker:** a bank detail change needs Finance approval, and payments are held until then. KYC approval and vendor activation need someone other than the person who invited the vendor.
- **Audit:** an append-only `audit_log` written in the same transaction as each change, covering every login, approval, status change and field edit. Retained 8 years.
- **Regulatory:** servers hosted in India; S3 and Textract in ap-south-1. DPDP Act 2023: consent notice at registration and a retention and deletion policy. The setup is reviewed against Solfin's RBI IT governance controls (quarterly access reviews, VAPT before go-live, change management). Documents sent to the parsing engine stay inside AWS ap-south-1, or the engine is self-hosted.
- **Infra:** Nginx rate limits on login and upload, TLS 1.2+, nightly pg_dump to S3 (30-day retention) with a tested restore, secrets in a `.env` file kept outside the repo, Postgres and Redis reachable only inside the compose network.

Integrations

Integration	Purpose	Phase	Approach
AWS S3	Document and invoice storage	MVP	Private bucket, SSE-KMS, lifecycle to Glacier after 1 year
Document parsing (Textract + LLM)	Pre-fill invoices and KYC fields	MVP	Worker job after upload; engine behind one interface
Email (Solfin custom email API)	All outbound notifications	MVP	Celery task calls the API; endpoint, auth

Integration	Purpose	Phase	Approach
			and payload format to be shared by Solfin
CSV / Excel export	Hand approved invoices and payments to Finance	MVP	Export from every list; scheduled daily file optional
KYC API provider	PAN verify, GSTIN lookup, PAN–name match	Phase 2	Background job on submit; provider TBD
Bank verification	Penny-drop on bank account	Phase 2	Same KYC provider or bank API
Entra ID	Internal SSO	Phase 2	OIDC
Data platform	Vendor and AP data into the warehouse	Phase 2	DMS / logical replication from Postgres into the existing AWS data stack
Accounting system	Book invoices and payments	Deferred	Out of scope for now; the export format is kept stable so an integration can be added later

Environments and deployment

Three environments, all running the same images via docker-compose and promoted by Git tag.

Env	Purpose	URL	Data
Local	Engineers, via `docker compose up`	localhost	Seed / fake data
UAT	Finance and approver sign-off, VAPT	vendor-uat.solfin.co.in	Masked copy, a few test vendors

Env	Purpose	URL	Data
Prod	Live	vendor.solfin.co.in	Live

Deployment shape for each env:

1. Hosting is managed by Solfin. The deliverable is a `docker/docker-compose.yml` that runs on any Docker host.
2. Compose services (one application image for `app`, `worker` and `beat`):
   - `nginx` (TLS, static files, proxy to `app`)
   - `app` (Django on gunicorn)
   - `worker` (Celery)
   - `beat` (Celery beat)
   - `redis`
   - `postgres` (named volume)
   - `clamav`
   - `backup` (nightly pg_dump to S3)
3. `docker-compose.test.yml` adds `minio` (S3-compatible) and a `playwright` runner, and sets `PARSER_ENGINE=fake` and `EMAIL_BACKEND=console`, so the full stack runs offline in CI.
4. Configuration lives in `.env`: Django secret key, DB credentials, S3 bucket and keys, Textract/LLM credentials, parser engine, email API URL and key, allowed hosts, CSRF trusted origins. `.env.example` is committed.
5. Release steps: `docker compose pull`, then `docker compose run --rm app python manage.py migrate`, then `docker compose run --rm app python manage.py collectstatic --noinput`, then `docker compose up -d`, then the Playwright smoke suite against the new environment (vendor login, upload + parse, L1 approve).
6. DNS at GoDaddy: A records for `vendor.solfin.co.in` and `vendor-uat.solfin.co.in` pointing at the host's IP. TLS via Certbot/Let's Encrypt in the nginx container, or Solfin's own certificate.
7. The coding agents generate the Django project, Dockerfile, docker-compose files, nginx config, seed data, Playwright suite and CI.

Sizing for start (assumed under 500 vendors and 3,000 invoices a month): one host with 4 vCPU, 8 GB RAM and 100 GB SSD for prod. Files live in S3, so local disk holds only Postgres data and logs.

Delivery plan and acceptance criteria

The MVP is about 10 weeks for a team of 2–3 Django engineers, 0.5 QA and 0.5 designer. These are estimates to confirm with the team. Playwright tests for each sprint's features are written in the same sprint, not after.

Sprint (2 wks)	**Deliverable**
1	Django project, base templates and layout, CI, docker-compose dev + test env, auth (login, OTP, reset), RBAC map, users and roles admin
2	Vendor invite, registration, KYC upload to S3, document checklist, KYC review, vendor states
3	Parsing pipeline (Textract + LLM) for invoices and KYC docs, parse UI with confidence highlights
4	Invoice submit with validations, approval matrix, Finance pre-check → L1 → L2 → L3 workflow, query thread
5	Payments, all MVP emails, reports and exports, audit log, UAT env, VAPT, prod go-live with 5 pilot vendors

Phase 2 (about 6 weeks) adds KYC/penny-drop APIs, SSO, SLA escalations, MSME 45-day tracking, document expiry and bulk upload. Phase 3 adds e-invoice QR validation, GSTR-2B reconciliation, bank payment files and WhatsApp.

MVP is done when:

- ☐ A vendor can go from invite to Active without emailing anyone
- ☐ Invoice parsing pre-fills at least 90% of header fields correctly on a 50-invoice test set from real vendors
- ☐ An invoice of ₹15 lakh routes Finance → L1 → L2 → L3 and stops at L1 for ₹50,000 (with the example limits)
- ☐ No user can approve an invoice they uploaded or approved at an earlier level
- ☐ The vendor A → vendor B isolation suite passes on every endpoint and file URL

- [ ] Every state change sends its email within 2 minutes and appears in the audit log
- [ ] VAPT has no open high or critical findings
- [ ] Restoring from backup in UAT has been tested and documented

Decisions and build defaults

Everything below is final for the build. Items marked *default* should be implemented as stated but kept configurable, so they can change without code changes.

Topic	**Decision**
Codebase	One Django 5 project (Python 3.12). No separate front end, no separate API service
UI	Django templates + Django forms, Tailwind (standalone CLI), HTMX for partial updates. No React/Next.js
ORM / migrations	Django ORM + Django migrations
Auth	Django session auth + email OTP; no JWT
Testing	pytest + pytest-django for unit tests; Playwright (pytest-playwright) for end-to-end, written in parallel with the code
Database	PostgreSQL 16
File storage	AWS S3 (private bucket, pre-signed URLs)
Jobs	Celery + Redis
Email	Solfin custom email API. Build a client behind an `EmailSender` interface with a console/log fallback until the API spec arrives
Domain	vendor.solfin.co.in (prod), vendor-uat.solfin.co.in (UAT); DNS at GoDaddy
Purchase orders	Not in v1. All invoices are non-PO and need a cost centre; the PO table is Phase 2
Accounting integration	Not in v1. CSV/Excel export only

Topic	Decision
Hosting	Managed by Solfin. Deliverable is docker-compose + Dockerfiles + nginx
Parsing engine	*Default:* AWS Textract (AnalyzeExpense / AnalyzeDocument) + LLM normalisation behind `parse_document(kind, file_id)`; engine chosen by env var
Approval limits	*Default:* L1 up to ₹1,00,000; L2 up to ₹10,00,000; L3 above. Set in the `approval_matrix` table
Finance step	*Default:* Finance pre-check before L1, then Finance again for payment
Approver assignment	*Default:* L1 = cost-centre owner; L2/L3 = pool by role, any member can approve, first action wins
Vendor Manager	*Default:* separate role; one person may also hold Finance, subject to maker-checker rules
Pilot vendors	To be named before UAT; not blocking the build